Introduction to Git & GitHub

Jennifer Graham

SAMS Session 2

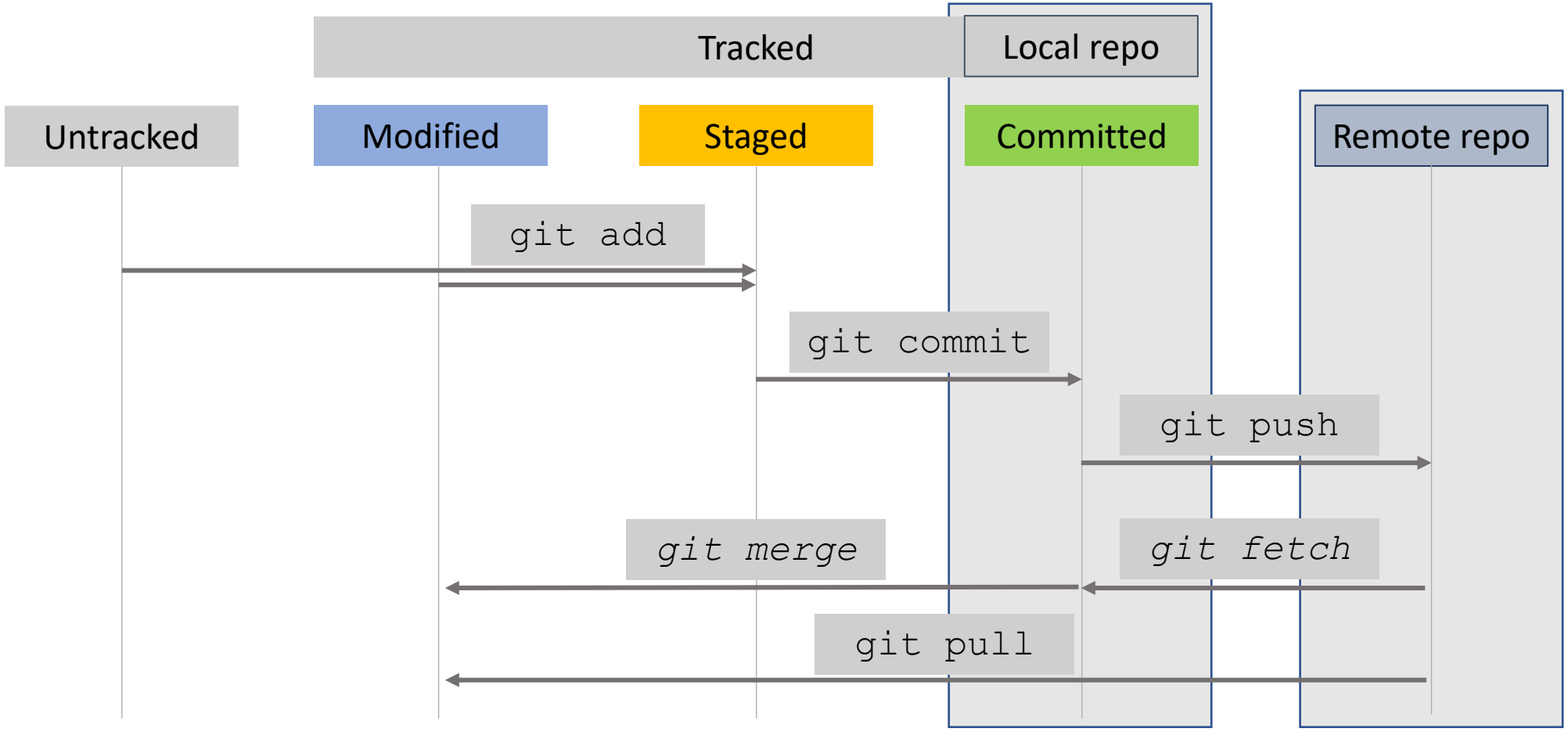
21st April 2026

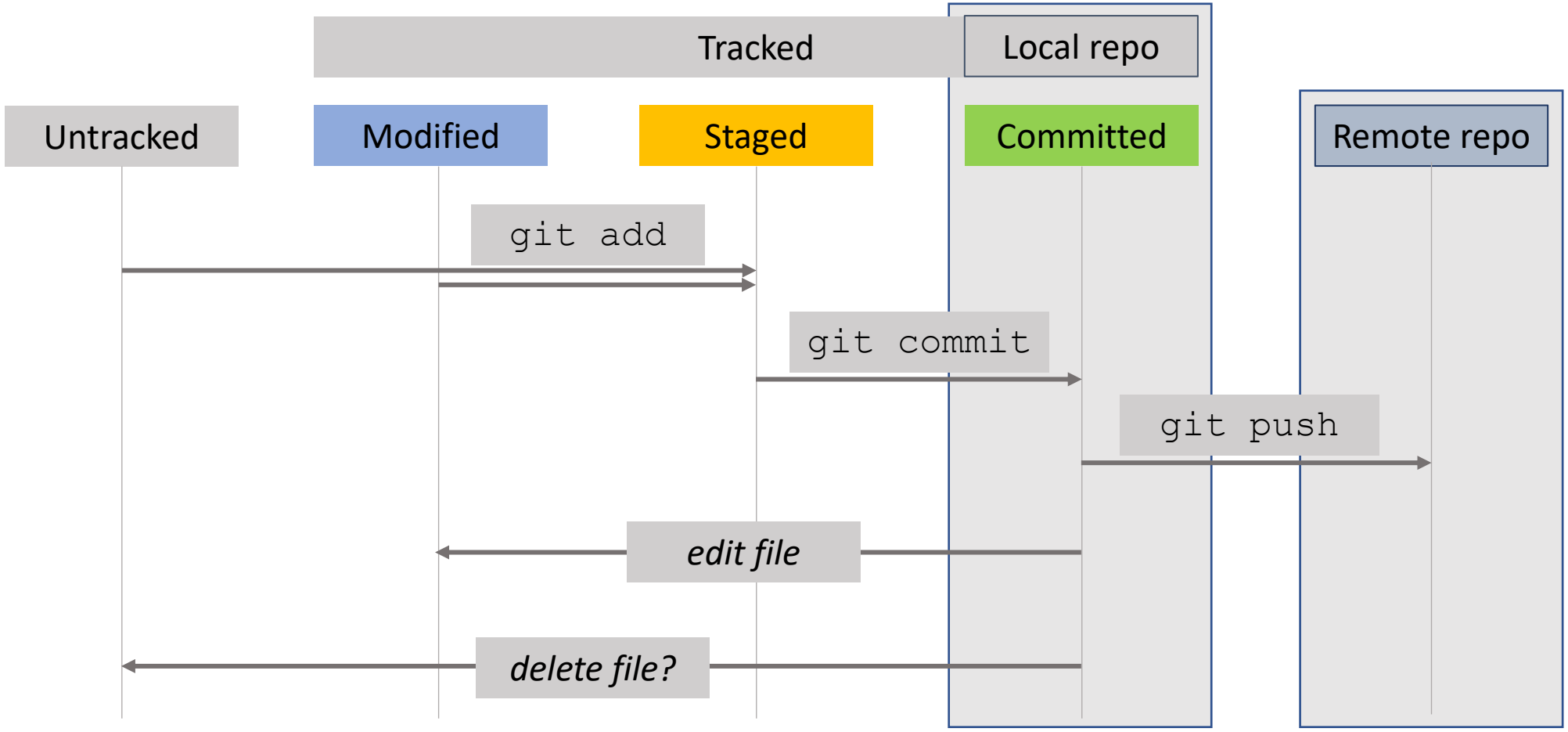
Recap from Session 1

Link to training materials here:

https://github.com/jenniferagraham/Git_Training

```
git init              :: turn current directory into git repository.
git add <file>        :: stage the file for commit i.e., track changes.
git commit            :: confirm changes with message/explanation.
git push              :: publish/back-up changes on GitHub (remote server).


git status            :: show which files have been tracked or modified.
git diff <file>       :: show difference between current file and last commit.
git log <file>        :: show commit history [for file]
```


Tracked
Local repo
Untracked
Modified
Staged
Committed
Remote repo
git add
git commit
git push
git merge
git fetch
git pull


Tracked
Local repo
Untracked
Modified
Staged
Committed
Remote repo
git add
git commit
git push
edit file
delete file?

Further useful commands…

```
git rm <file>                :: delete file and remove from repository.
git rm --cached <file>       :: delete file from repository, but keep
                                local copy.

git mv <file> <new_name>     :: rename file, and keep tracking.


git <command> --help         :: open help documentation for any command!
```

Managing your own projects

When mistakes happen…

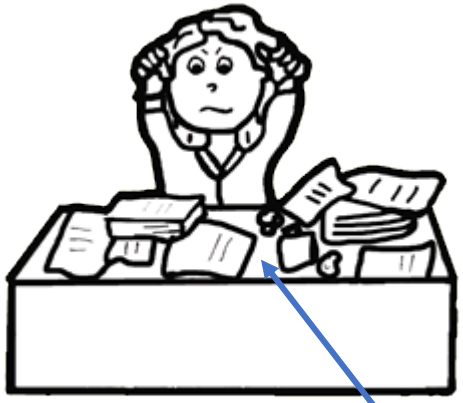
Contents of `.git` folder

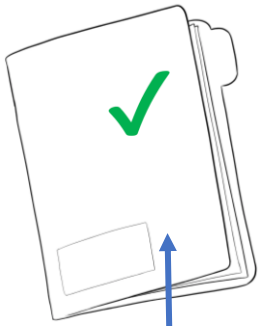
- When a repository is created, this creates a .git folder in the directory.
- .git contains e.g.:

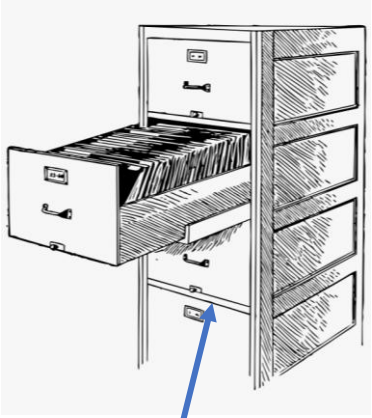
```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$ ls -a
./  ../  .git/  README.md  hello_world.py

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$ ls .git
COMMIT_EDITMSG  FETCH_HEAD  HEAD  ORIG_HEAD  config  description  hooks/  index  info/  logs/  objects/  refs/
```

- What are these files?
  - They contain everything that git needs to know about the repository!
  - Whenever you run a command, it will use and/or edit files here.

How does git work?







Working directory	Staging area	Commit tree
Messy!	Changes to include at next save	Record of all the saved versions

Different commands modify different states of files in your local directory.

"I will want to record changes to this document at the next save"

`add` file to staging area

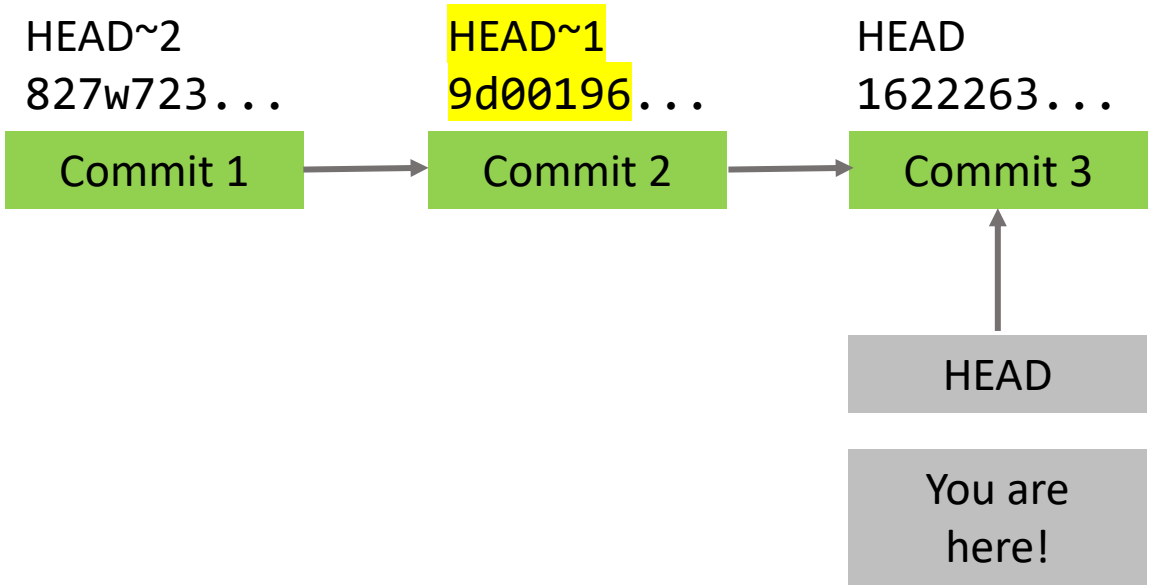
"Save all the files marked"

`commit` (staged files)

"Get me yesterday's version of this file"

`checkout` yesterday's file…

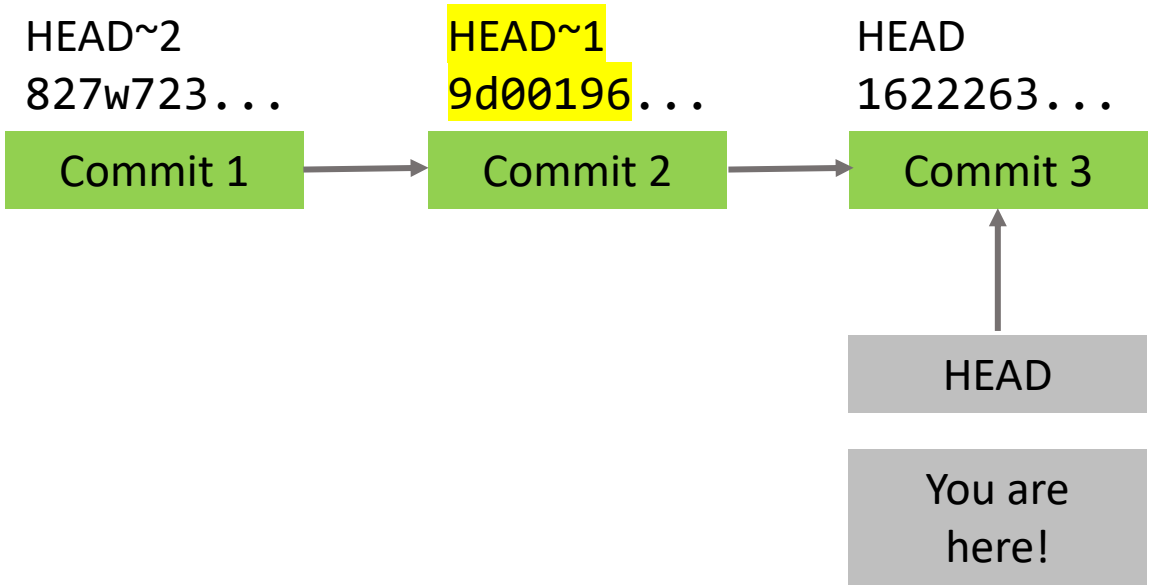
How to refer to a `commit` (version)



How to refer to a `commit` (version)

Relative to the last commit

- HEAD is last commit
- HEAD~*n* is *n* before last



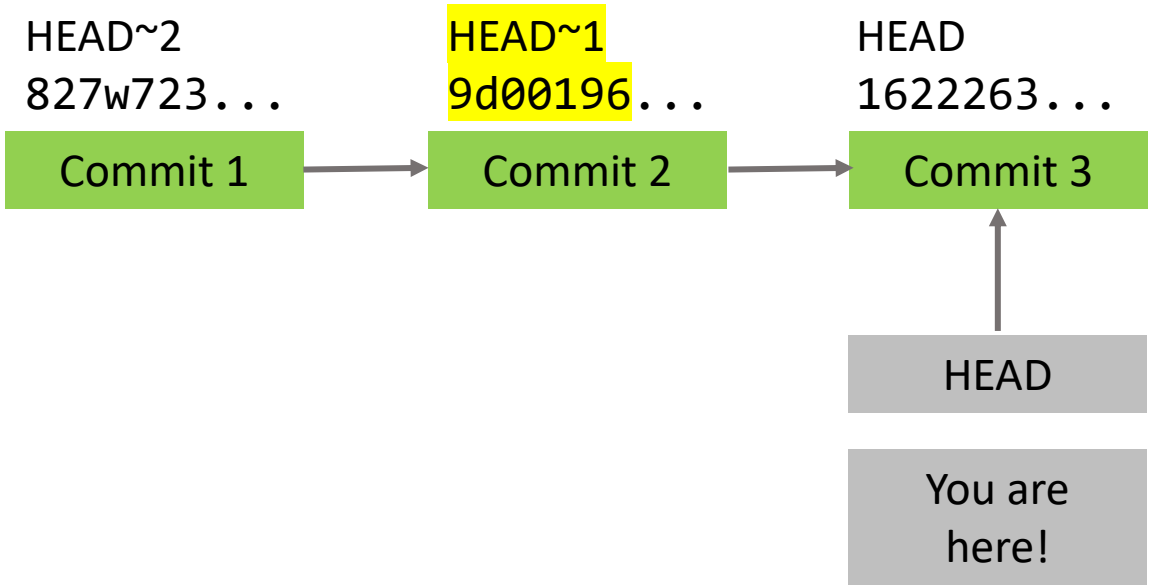
How to refer to a `commit` (version)

Relative to the last commit

- HEAD is last commit
- HEAD~*n* is *n* before last

Absolute reference

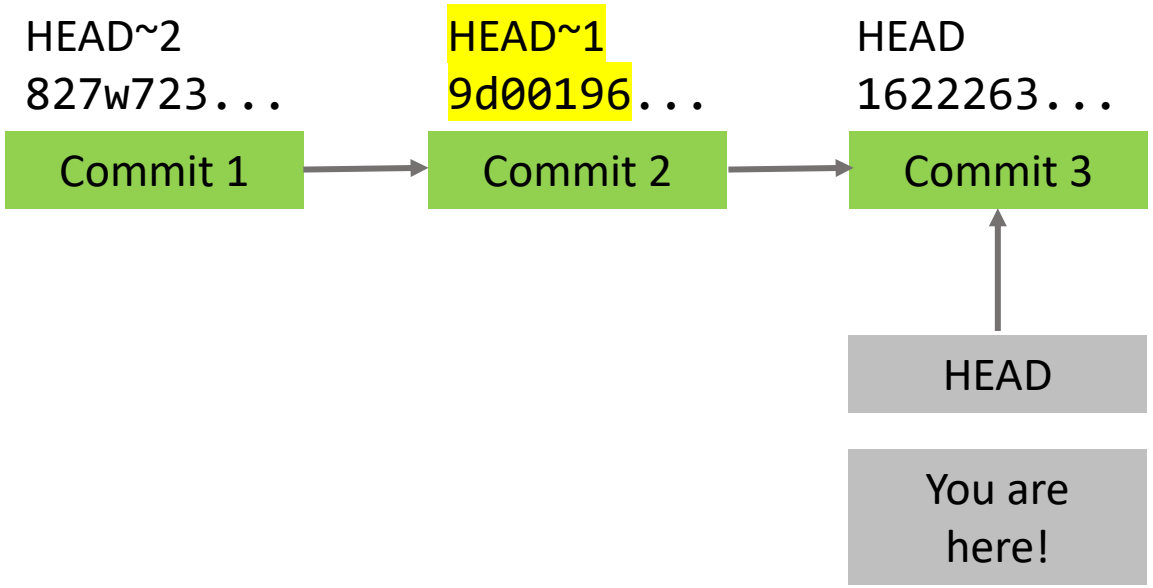
- use a “hash” from `git log`



How to refer to a `commit` (version)

Relative to the last commit

- HEAD is last commit
- HEAD~*n* is *n* before last



Absolute reference

- Use a “hash” from `git log`
- The first 10 characters are usually enough

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$ git log
commit 01b729dee53142b87c85a6802f5f4cc3df38c9a2 (HEAD -> main)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Mar 30 16:48:14 2026 +0100

    Fixing typo

commit e2c5dde1359e5a03477dc8c30fb2e424c879bcf4
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Mar 30 16:23:56 2026 +0100

    Adding README file

commit d63f8999c47f376c62ca293caedd439697b2cd9c
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Mar 30 16:16:23 2026 +0100

    Adding first file to repository
```

How to restore previous version of file(s)?

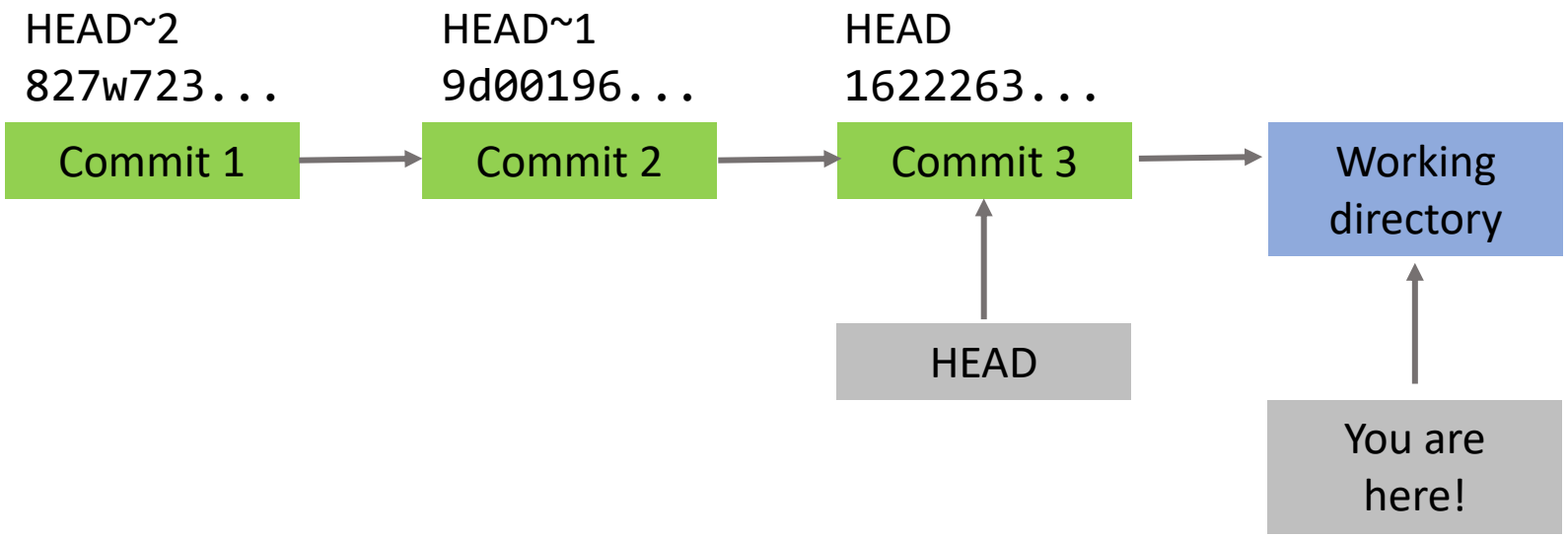
```
git checkout [<commit>] [--] <path>
:: retrieve version of file(s) from specified commit.
   NB. "--" just means staying in the same branch.
```

When specify path, used to restore previous <commit> version of file(s).

Without specified <path>, can be used to explore alternative state of entire repository…
[more on this later]

Working directory	Staging area	Commit tree
Yes	**Yes**	No

How to discard changes since last commit?




```
git checkout <file>
:: retrieve last committed version [for file]
```

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$ git checkout README.md
Updated 1 path from the index

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

```
git checkout [--] <file>
:: discard changes in working directory [for file]
```

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$
```

```
git restore <file>
:: discard changes in working directory [for file]
```

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$ git add README.md

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/hello_world (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
```

```
git restore --staged <file>
:: to unstage specified file for commit
```

How to undo a “bad” commit?

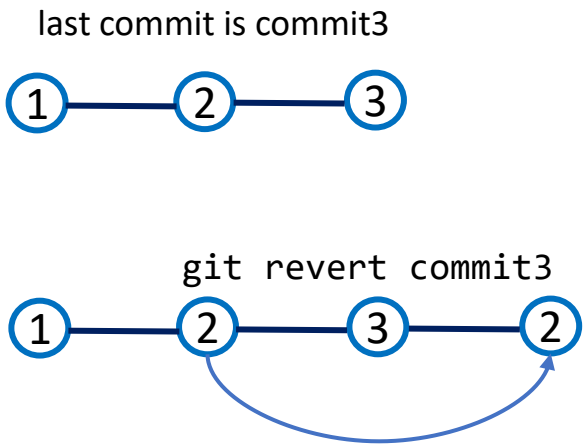
- There are multiple ways to roll back changes in your code, e.g.

```
git checkout <commit> [--] <path>
:: retrieve version of file(s) from specified commit.

git revert <commit>
:: undo your chosen commit, keeping record of change.

git reset [--soft/hard] <commit>
:: discard all commits [and possibly changes] since chosen version
      - use with care!!!
```

Undo changes of single commit

```
git revert HEAD
:: undo your most recent commit, keeping record of change.
```

- Generate a new commit reversing the changes made in last commit
- Maintains a record of the offending commit in the history



Working directory	Staging area	Commit tree
Yes	No	**Yes (preserving history)**

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git log
commit f5bcc48ce0534afcbcd289a8c1725059e5a2c0ca (HEAD -> main)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:48:08 2026 +0100

    Adding new file

…
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ ls
newfile.txt  testfile.txt

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git revert HEAD
hint: Waiting for your editor to close the file... unix2dos: converting file
C:/Users/sa04jg/OneDrive - SAMS/GitHub/Repos/git_testing/.git/COMMIT_EDITMSG to DOS format...
dos2unix: converting file C:/Users/sa04jg/OneDrive -
SAMS/GitHub/Repos/git_testing/.git/COMMIT_EDITMSG to Unix format...
[main 78c6862] Revert "Adding new file"
 1 file changed, 1 deletion(-)
 delete mode 100644 newfile.txt

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ ls
testfile.txt
```

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ ls
testfile.txt

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git log
commit 78c68629d580bbcc77b57b886fc0215c7186ecaf (HEAD -> main)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:48:51 2026 +0100

    Revert "Adding new file"

    This reverts commit f5bcc48ce0534afcbcd289a8c1725059e5a2c0ca.

commit f5bcc48ce0534afcbcd289a8c1725059e5a2c0ca
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:48:08 2026 +0100

    Adding new file

commit 3c5ceae433e0d68307e94ace364c064333abe7e3
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:18:47 2026 +0100

    4th commit
…
```

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ ls
testfile.txt

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git log
commit 78c68629d580bbcc77b57b886fc0215c7186ecaf (HEAD -> main)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:48:51 2026 +0100

    Revert "Adding new file"


…
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git revert 78c68629d580bbc
hint: Waiting for your editor to close the file... unix2dos: converting file
C:/Users/sa04jg/OneDrive - SAMS/GitHub/Repos/git_testing/.git/COMMIT_EDITMSG to DOS format...
dos2unix: converting file C:/Users/sa04jg/OneDrive -
SAMS/GitHub/Repos/git_testing/.git/COMMIT_EDITMSG to Unix format...
[main fbef154] Reapply "Adding new file"
 1 file changed, 1 insertion(+)
 create mode 100644 newfile.txt

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ ls
newfile.txt  testfile.txt
```

Undo changes of single commit

```
git revert <commit>
:: undo your chosen commit, keeping record of change.
```

- NB. Can only revert **independent** commits, e.g.,
  - What would happen if we try to revert creation of a file, or individual line, that has further changes?

Committing associated changes and often makes it easier to roll back mistakes, but won't always avoid conflicts...

Undo [and discard] changes *since* your chosen commit

```
git reset [--soft/hard] <commit>
:: --soft discard all commits since your chosen commit version
:: --hard discard all commits AND discard record of changes in work directory.
```

- **Use with care!** Both “soft” and “hard” versions discard commits without any record of reason.

- **Soft** version: useful to undo a recent commit that was a genuine mistake while retaining changes in working directory.
- **Hard** version: discards all commits AND edits without any record of reason... can lose files or edits.

Working directory	Staging area	Commit tree
Soft: No Hard: Yes	Yes	Yes

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ ls
newfile.txt  testfile.txt

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git log
commit f5bcc48ce0534afcbcd289a8c1725059e5a2c0ca (HEAD -> main)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:48:08 2026 +0100

    Adding new file
…
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git reset --soft HEAD~1

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git log
commit 3c5ceae433e0d68307e94ace364c064333abe7e3 (HEAD -> main)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:18:47 2026 +0100

    4th commit
…
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ ls
newfile.txt testfile.txt
```

Accidental commit – added newfile.txt before testing?

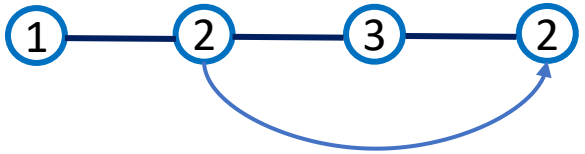
“Soft” reset removed the commit, but kept the file.

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ ls
newfile.txt  testfile.txt

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git log
commit f5bcc48ce0534afcbcd289a8c1725059e5a2c0ca (HEAD -> main)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:48:08 2026 +0100

    Adding new file
…
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git reset --hard HEAD~1

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git log
commit 3c5ceae433e0d68307e94ace364c064333abe7e3 (HEAD -> main)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:18:47 2026 +0100

    4th commit
…
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ ls
testfile.txt
```

“Hard” reset removed the commit and the file!

```
git restore <path>
:: discard changes in working directory for chosen file(s).

git checkout <commit> [--] <path>
:: retrieve version of file(s) from chosen commit.

git revert <commit>
:: undo your chosen commit, keeping record of change.

git reset [--soft/hard] <commit>
:: discard all commits [and possibly changes!] since chosen version
      - use with care!!!
```

git revert commit3



git reset --soft/hard commit2

Try it yourself… [20 min]

```
Reminder
git init
git add <file>
git commit
```

1. Create your own local repository for testing:
   a) Add and commit a new file, e.g., named “newfile.txt”.
   b) Commit 4 changes to the file, e.g., each adding a new line of text.

2. Use “`git checkout`” to retrieve earlier versions of the file.
   a) Notice what happens in your editor after you run each command.
   b) Retrieve the latest version before moving to next step.

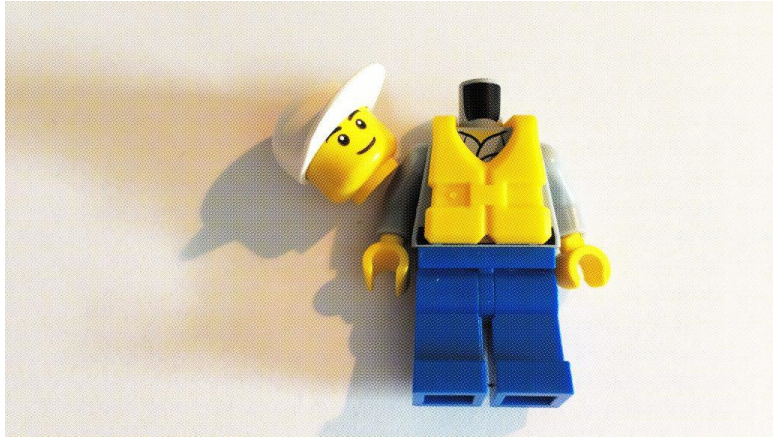
3. What command(s) would you use to do the following?
   a) Edit your file, discard changes in working directory.
   b) Undo the most recent commit, keeping record of decision.
   c) Undo the past 2 commits BUT keep changes in your working directory.

Try the above!

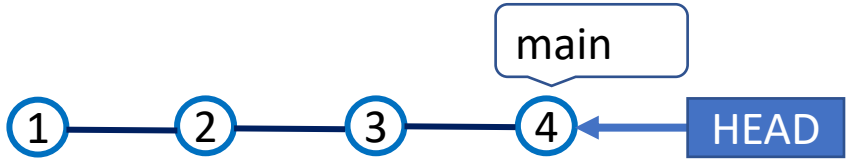
At each step, use `git log`, `git status` and `git diff`, to explore changes in your files and commit history (or get some hints?).

BONUS question: What happens if you try to “`revert`” your first commit…?

Detached HEAD state …?

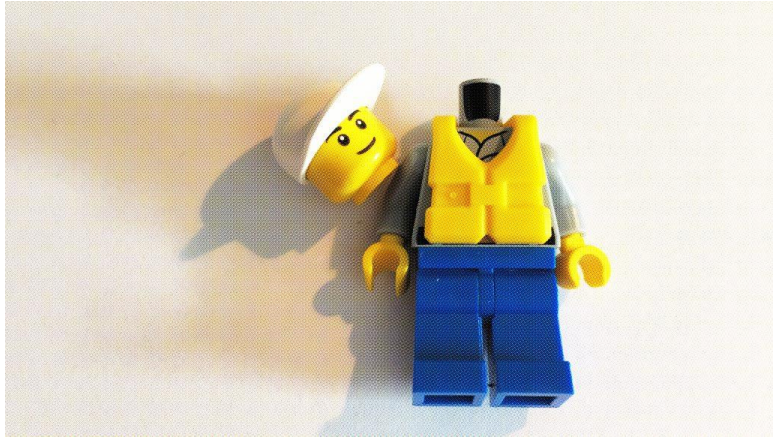


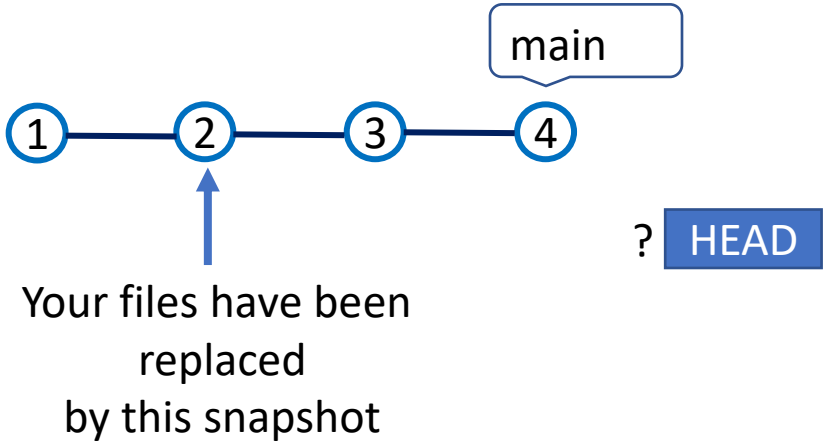
- Reminder: HEAD is a pointer that refers to the end of the branch*
- When you checkout an intermediate commit, the HEAD gets “detached”



Detached HEAD state …?

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git checkout HEAD~2
Note: switching to 'HEAD~2'.

You are in 'detached HEAD' state. You can look around, make
experimental changes and commit them, and you can discard any
commits you make in this state without impacting any branches by
switching back to a branch.
…

HEAD is now at 6becff0 "2nd commit"

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing ((6becff0...))
$ git status
HEAD detached at 6becff0
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        newfile.txt

nothing added to commit but untracked files present (use "git add"
to track)
```

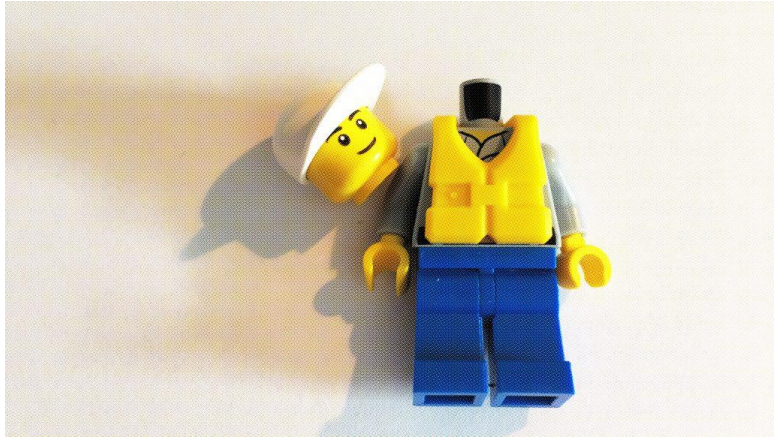


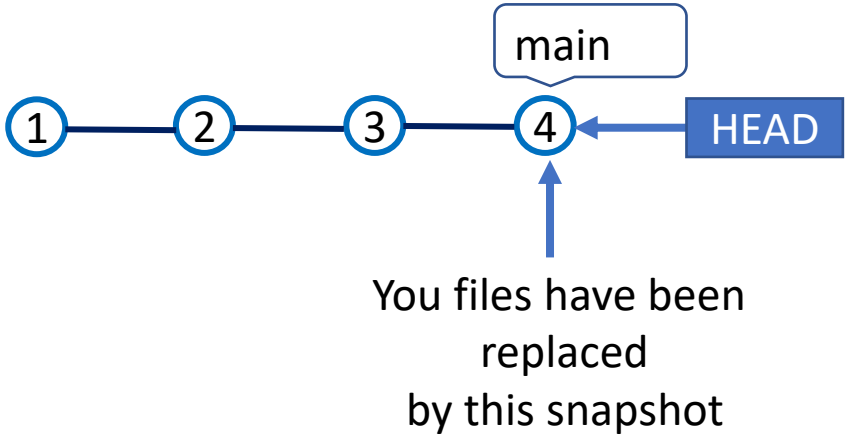


This is a normal state if you are <u>inspecting past commits</u>, but warning that any changes or commit made here can be easily lost.

Detached HEAD state …?

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git checkout HEAD~2
Note: switching to 'HEAD~2'.

You are in 'detached HEAD' state.
…
Or undo this operation with:

 git switch -
…

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing ((6becff0...))
$ git switch -
Previous HEAD position was 6becff0 2nd commit
Switched to branch 'main'

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$

OR

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing ((6becff0...))
$ git checkout main
Previous HEAD position was 6becff0 2nd commit
Switched to branch 'main'
```





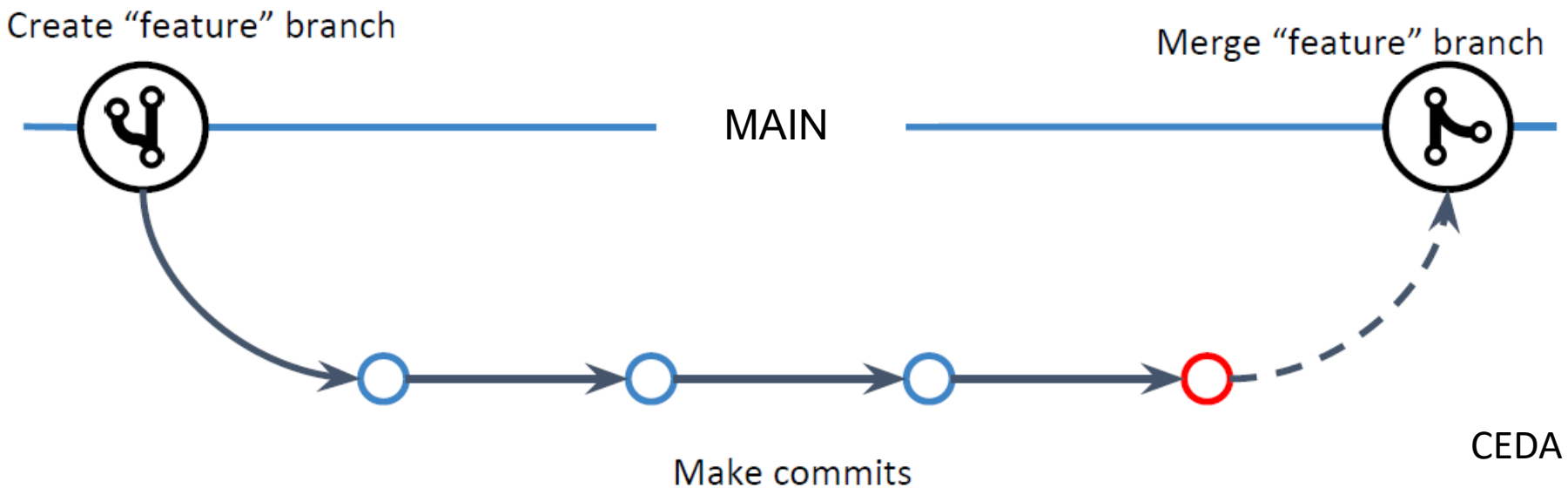
Return to orderly state by checking out the most recent commit of your branch.

Branching and Merging

- What is a branch?
- When should I create a branch?
- How do I merge changes?

Branching

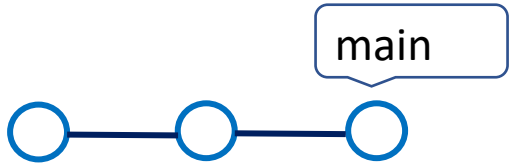
- The main branch should always work.
- To test experimental changes or new features, you can create a branch.
- A new branch creates a new end point (HEAD) in your chain of commits.



CEDA

Branching

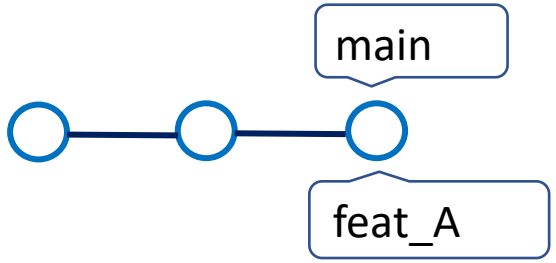
- Branches are pointers to an end commit



Note: main is just the default name for the first branch (can rename it!)

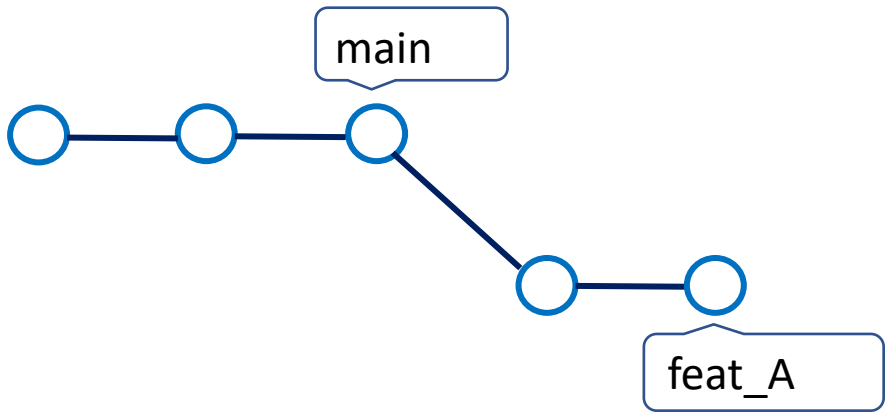
Branching

- Branches are pointers to an end commit
- By creating branch, e.g., feat_A, a new end point is created



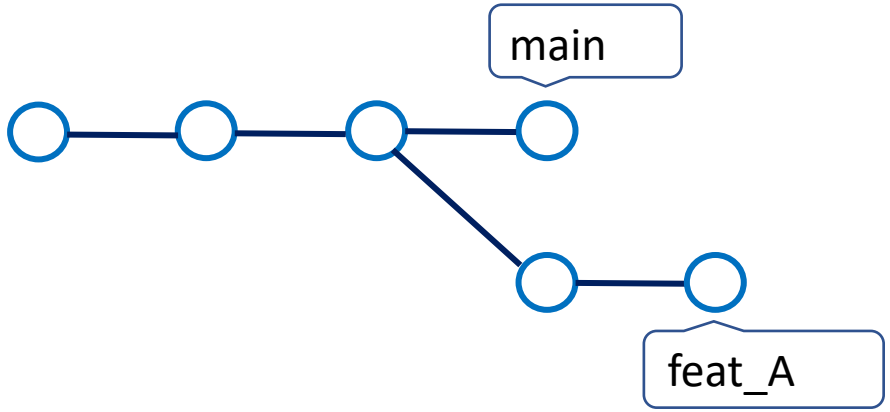
Branching

- Branches are pointers to an end commit
- By creating branch feat_A new end point is created
- New commits in feat_A won't affect main



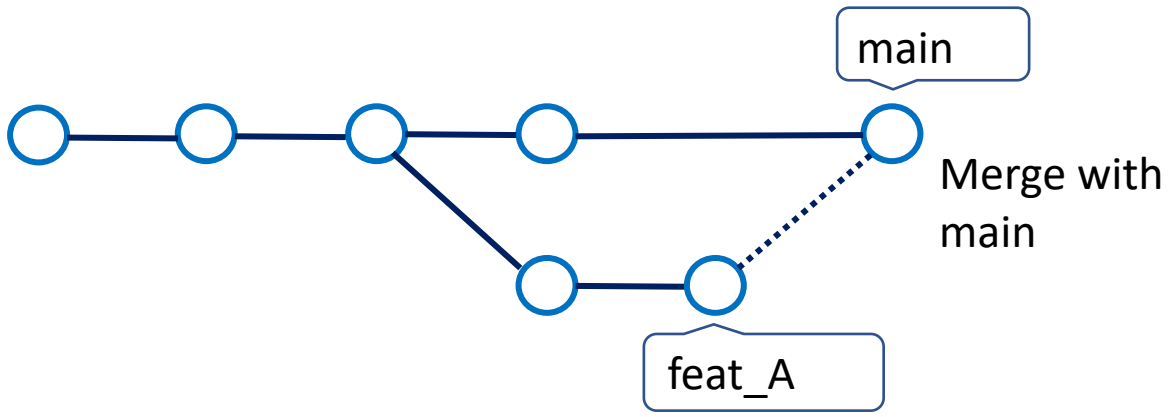
Branching

- Branches are pointers to an end commit
- By creating branch feat_A new end point is created
- New commits in feat_A won't affect main
- And new commits in main won't affect feat_A



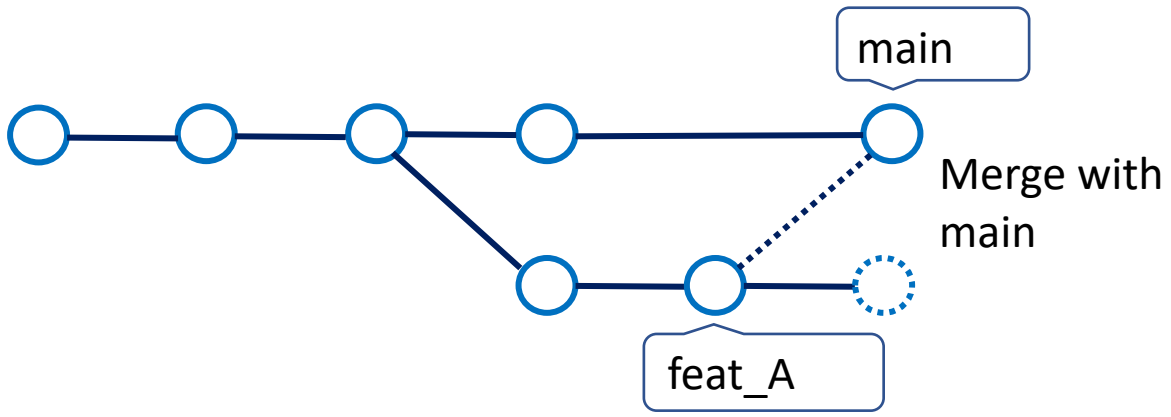
Branching

- When feat_A is ready, it can be merged with main



Branching

- When feat_A is ready, it can be merged with main
- feat_A might be deleted or continue as a persistent branch



How to create & move between branches?

```
git branch    :: list existing branches; * = you are here.

git branch <branch-name>  :: create a new branch, <branch-name>.

git checkout <branch-name>     :: "switch" onto branch,
                                     <branch-name>.
git switch [-c] <branch-name> :: switch to your branch,
                                 <branch-name>,
                                 -c option = create if new
                                 (shortcut for git branch and checkout).

git switch - :: shortcut to return to previous branch.
```

Example?

- List existing branches
  - * = you are here
- Create branch *text*
- Using `git branch`
  & `git checkout`

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git branch
* main

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git branch text

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git branch
  text
* main

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git checkout text
Switched to branch 'text'

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (text)
$
```

Example?

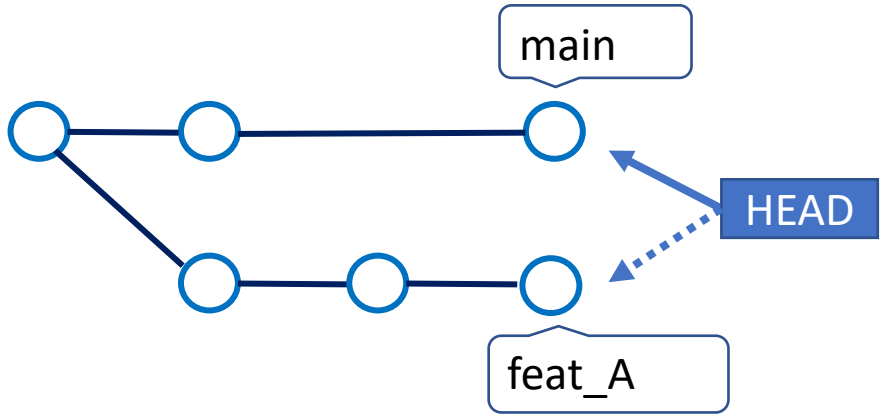
- List existing branches
  - * = you are here
- Create branch *text*
- Using `git branch`
  & `git checkout`

OR

`git switch -c …`

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git branch
* main

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git switch -c text
Switched to a new branch 'text'

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (text)
$ git branch
  main
* text
```

git switch <branch>



New alternative to “checkout”.

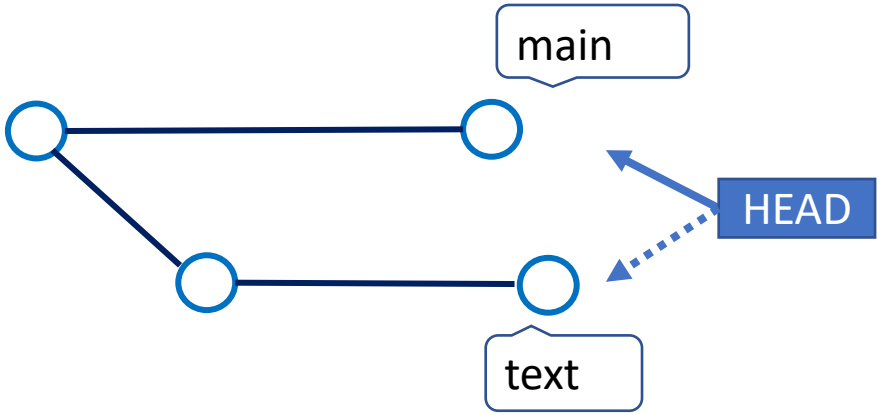
Moves HEAD pointer to <branch>

The files in the working area will be changed accordingly.

NB. After switching, just use the same core workflow and new commits will be added to the new branch.

Can show an example…

git switch <branch>



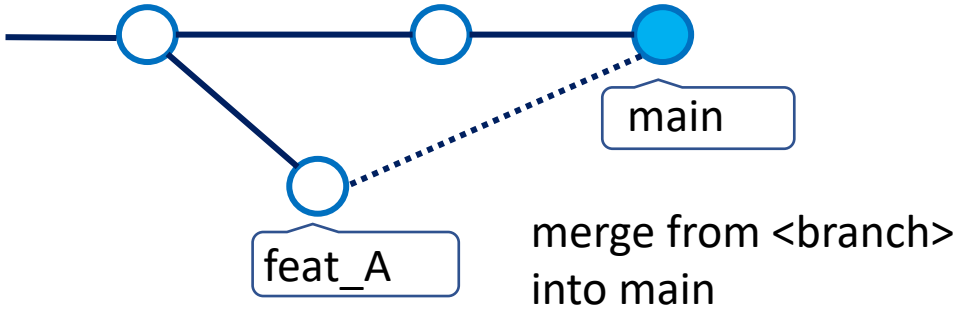

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing
(main)
$ git log
commit 6f8e2f93d6c5ab0dad124d99fa06457f75df7620
(HEAD -> main)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 20:37:51 2026 +0100

    Modify 4

commit 3c5ceae433e0d68307e94ace364c064333abe7e3
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:18:47 2026 +0100

    4th commit
…
```

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing
(text)
$ git log
commit 55532e5f01c14a0e15a975ce0d6871ec48d2354f
(HEAD -> text)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 20:35:58 2026 +0100

    sixth commit

commit 218fddbc2487ea8ce66e73a61593402449ac0bd3
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 20:35:21 2026 +0100

    Fifth commit

commit 3c5ceae433e0d68307e94ace364c064333abe7e3
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:18:47 2026 +0100

    4th commit
…
```

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (text)
$ git switch main
newfile.txt  testfile.txt

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git diff main..text
diff --git a/testfile.txt b/testfile.txt
index d5fdd3c..d201875 100644
--- a/testfile.txt
+++ b/testfile.txt
@@ -1,4 +1,5 @@
 Commit 1
 2
-3
-4
+3 4
+five
+six

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$
```

```
git diff <branch1>..<branch2> <path>
:: show difference between branches, for optional file(s) <path>
```

Merge branches



- When happy with changes, can choose to merge between branches.
- Switch to main (or other branch you want to **merge changes into**).

- Merges changes from your chosen branch:

```
git merge <branch-name>    :: apply new commits from <branch-name>
                                (since split) onto present branch.
```

Git can merge automatically when changes are:
- in separate files
- on separate locations (i.e., lines) of the same file

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (text)
$ git switch main
Switched to branch 'main'

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git diff main..text
diff --git a/testfile.txt b/testfile.txt
index d5fdd3c..d201875 100644
--- a/testfile.txt
+++ b/testfile.txt
@@ -1,4 +1,5 @@
 Commit 1
 2
-3
-4
+3 4
+five
+six

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git merge text
```

Do you think this will merge automatically?

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git diff main..text
diff --git a/testfile.txt b/testfile.txt
index d5fdd3c..d201875 100644
--- a/testfile.txt
+++ b/testfile.txt
@@ -1,4 +1,5 @@
 Commit 1
 2
-3
-4
+3 4
+five
+six

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git merge text
Auto-merging testfile.txt
CONFLICT (content): Merge conflict in testfile.txt
Automatic merge failed; fix conflicts and then commit the result.

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main|MERGING)
$
```

To resolve a conflict, need to edit the resulting file and commit…

Conflict resolution

When merge doesn't come easy…

- If there are conflicting changes on the same line(s)?

After the failed merge

```
Commit 1
2
<<<<<<< HEAD
3
4
=======
3 4
five
six
>>>>>>> text
```

After your edit

```
Commit 1
2
3
4
five
six
```

- <<< === and >>> separate the two versions.
- You choose which lines / edits you want to keep.

Once resolved: save, add and commit

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git merge text
Auto-merging testfile.txt
CONFLICT (content): Merge conflict in testfile.txt
Automatic merge failed; fix conflicts and then commit the
result.

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing
(main|MERGING)
$ git add testfile.txt

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing
(main|MERGING)
$ git commit
hint: Waiting for your editor to close the file...
[main 9db6f54] Merge branch 'text' Resolving conflicts.

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git log
…
```

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git log
commit 9db6f54532b70c4f4de8668cc8782759dafdc682 (HEAD -> main)
Merge: 6f8e2f9 55532e5
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 21:18:42 2026 +0100

    Merge branch 'text'
    Resolving conflicts.

commit 6f8e2f93d6c5ab0dad124d99fa06457f75df7620
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 20:37:51 2026 +0100

    Modify 4

commit 55532e5f01c14a0e15a975ce0d6871ec48d2354f (text)
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 20:35:58 2026 +0100

    sixth commit

commit 218fddbc2487ea8ce66e73a61593402449ac0bd3
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 20:35:21 2026 +0100

    Fifth commit

commit 3c5ceae433e0d68307e94ace364c064333abe7e3
Author: Jennifer Graham <jennifer.graham@sams.ac.uk>
Date:   Mon Apr 20 10:18:47 2026 +0100

    4th commit

. . .
```

```
Merged testfile.txt:

Commit 1
2
3
4
five
six
```

```
git branch    :: list existing branches; * = you are here.

git branch <branch-name>    :: create a new branch, <branch-name>.
git checkout <branch-name> :: "switch" onto branch, <branch-name>.

git switch [-c] <branch-name>
     :: switch to your branch, <branch-name>,
     -c option = create if new (shortcut for git branch and checkout).

git merge <branch-name>
     :: apply new commits from <branch-name> (since split), onto present branch

git diff <branch1>..<branch2> [<path>]
     :: check difference between two branches, for optional file(s) <path>
```

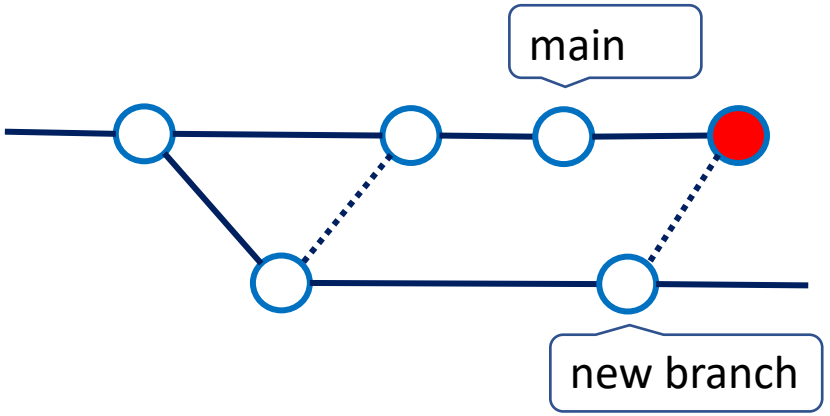
Try it yourself (15 mins):

throw a spanner in the works challenge

1. Create a new branch in a repository with 2 files

Simple merge

2. On file A, create a commit on only the new branch; checkout main and merge your new branch to main



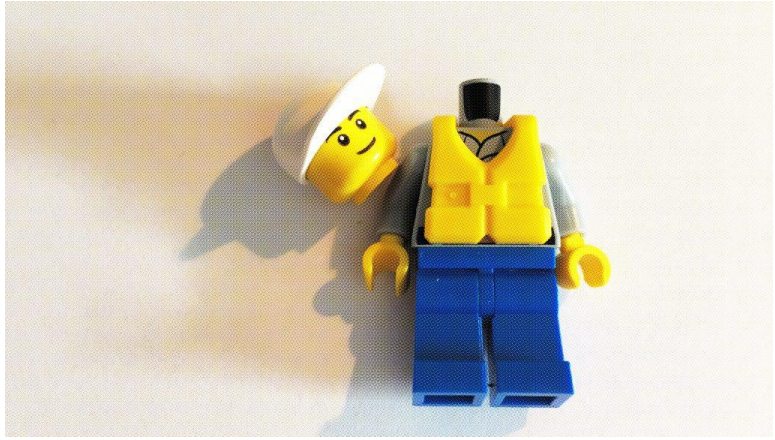
Break the automatic merge

3. Create commits in file B in both branches and try to cause a merge conflict.

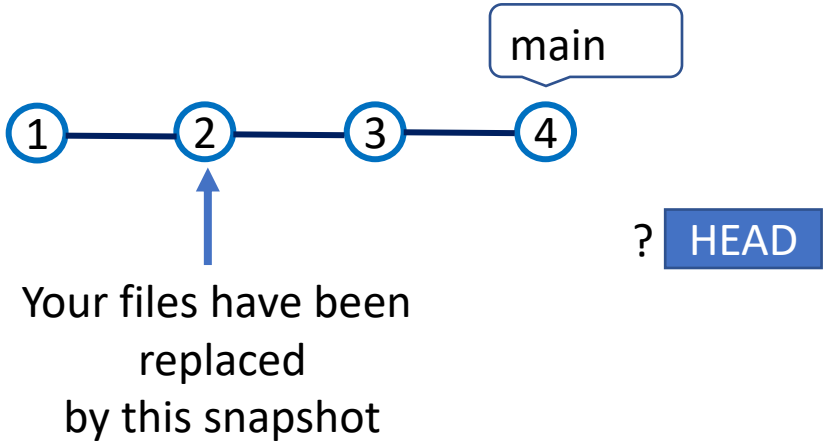
4. Now, sort out the mess...

More useful tips…

Remember detached HEAD state …?



```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git checkout HEAD~2
Note: switching to 'HEAD~2'.

You are in 'detached HEAD' state. You can look around, make
experimental changes and commit them, and you can discard any
commits you make in this state without impacting any branches by
switching back to a branch.

If you want to create a new branch to retain commits you create,
you may do so (now or later) by using -c with the switch command.
Example:

  git switch -c <new-branch-name>

…

HEAD is now at 6becff0 "2nd commit"

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing ((6becff0...))
$ git status
HEAD detached at 6becff0
…
```



Option to branch from a previous point in time?

git stash

- git stops you from checking out when you have local modifications

- If you are not yet ready to commit those modifications, you can *git stash* them away for use later

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing
(main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be
committed)
  (use "git restore <file>..." to discard changes in
working directory)
        modified:   testfile.txt

Untracked files:
  (use "git add <file>..." to include in what will be
committed)
        newfile.txt

no changes added to commit (use "git add" and/or "git
commit -a")

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing
(main)
$ git checkout text
error: Your local changes to the following files would
be overwritten by checkout:
        testfile.txt
Please commit your changes or stash them before you
switch branches.
Aborting
```

`git stash`

- git stops you from checking out when you have local modifications

- If you are not yet ready to commit those modifications, you can *git stash* them away for use later

1. `git stash`
2. `git checkout / switch` to your heart's content
3. `git stash apply` to restore the working directory and staging area

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git checkout text
error: Your local changes to the following files would be overwritten by checkout:
        testfile.txt
Please commit your changes or stash them before you switch branches.
Aborting

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git stash
Saved working directory and index state WIP on main: 9db6f54 Merge branch 'text' Resolving conflicts.

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git switch text
Switched to branch 'text'

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (text)
$ git switch main
Switched to branch 'main'

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        newfile.txt

nothing added to commit but untracked files present (use "git add" to track)
```

```
JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        newfile.txt

nothing added to commit but untracked files present (use "git add" to track)

JG@SAMS MINGW64 ~/OneDrive/GitHub/Repos/git_testing (main)
$ git stash apply
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   testfile.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        newfile.txt

no changes added to commit (use "git add" and/or "git commit -a")
```